Research Square

Preprints are preliminary reports that have not undergone peer review. They should not be considered conclusive, used to inform clinical practice, or referenced by the media as validated information.

Infusing Syntax and Semantics into LLMs

Anton Bulle Labate

antonlabate@usp.br

Center for Artificial Intelligence and Machine Learning

Fabio Gagliardi Cozman

Center for Artificial Intelligence and Machine Learning

Research Article

Keywords: Text2SQL, Multilinguality, Semantics, LLMs

Posted Date: July 15th, 2025

DOI: https://doi.org/10.21203/rs.3.rs-6958223/v1

License: This work is licensed under a Creative Commons Attribution 4.0 International License. Read Full License

Additional Declarations: No competing interests reported.

Infusing Syntax and Semantics into LLMs

Anton Bulle Labate and Fabio Gagliardi Cozman

Center for Artificial Intelligence and Machine Learning, Universidade de São Paulo, Av. Prof. Lúcio Martins Rodrigues, 370, São Paulo, 05508-020, São Paulo, Brazil.

Contributing authors: antonlabate@usp.br; fgcozman@usp.br;

Abstract

Despite impressive success, large language models (LLMs) sometimes generate outputs with flawed linguistic structure. We analyze the effect of directly infusing various kinds of syntactic and semantic information into LLMs. To demonstrate the value of our proposals, we focus on the translation of natural language queries to SQL — in particular dealing with languages with less resources than English, to better investigate how much help we can get from low-cost syntactic and semantic information. We show that linguistic analysis can significantly boost language models and even surpass existing best systems for some languages.

Keywords: Text2SQL, Multilinguality, Semantics, LLMs

1 Introduction

In this paper, we introduce techniques that resort to syntactic and semantic information to improve the performance of language models. Intuitively, one would expect that solid linguistic knowledge and honed linguistic tools should be useful in natural language processing, in particular in preventing a number of failure modes that have been observed even in large language models [1, 2].

Here we explore this intuition, and we indeed find that syntactic and semantic information are rather valuable, at least in settings where existing data-driven resources are limited by practical or economic circumstances. In essence, we infuse syntactic and semantic information into textual prompts and during fine-tuning procedures, instead of changing computational paths as investigated in previous proposals.

To be concrete, we focus our experiments on the translation of natural language questions to SQL queries; our proposals should be easily transferable to other tasks

in natural language processing. We focus on languages other than English, where the quality of query translation has evolved at a lower pace than it has for English, so as to face more challenging scenarios that can shed light on the value of linguistic devices. We show that, at a relatively low cost, one can significantly boost the performance of language models — for instance, we have been able to surpass the previous state of the art on the versions of the Spider benchmark for Portuguese and French queries. We hope these results will drive people to revisit syntactic and semantic information that can have as much effect as large amounts of data while consuming less resources. In fact, there seems to be a trade-off: one can make up for limited data/computing resources by resorting to linguistic tools, and vice-versa — a huge language model may benefit less from linguistic information as it may already internally represent linguistic tools learned from data.

The paper is organized as follows. In Section 2 we briefly go through relevant previous work; in Section 3 we describe our proposals, and in Section 4 we describe our experiments. Section 5 wraps up with some discussion and conclusions.

2 Background and Related Work

There are a number of techniques that aim at infusing morphological/syntactic/semantic information into language models (LMs). Punyakanok et al. (2008) [3] is an early example of syntactic infusion in role labeling (without transformers). Other efforts mix classic linguistic results within learning techniques [4]. Shiv and Quirk (2019) [5] use a positional embedding that exploits relations in a syntax tree, so as to get some of the syntactic structure into an LM. Clearly this requires changes to the original transformer model. Another effort, by Qian et al. (2021) [6], modify the training of the LM by asking the model to predict not only words but also parse trees.

Morphological parse trees have also been used during training; Bai et al. (2021) [7] have self-attention modules for each relation from a morphological tree. And morphological trees have been mapped to vectors by embeddings [8], with tests on encoder-only models for classification tasks. Currey and Heafield (2019) [9] enhance translation performance by infusing morphology trees as input to trained encoders, and also improve translation with encoder-decoder models by pre-training the model on a parsing task. Zhang et al. (2020) [10] use a network to insert morphological relations into the self-attention mechanism of a LM. Regarding the transformer's attention mechanism, Zhang et al. (2022) [11] use the information of the words related to each other in the attention mechanism.

Structures beyond morphological relations have also been infused into LMs. For instance, Yasunaga et al. (2022) [12] propose a self-supervised and bidirectional deep pre-training method using texts and knowledge graphs, and a self-supervised bidirectional joint generation objective that learns joint reasoning over text and graph. Also exploring the use of different phrase's structures, Zhang et al. (2020) [13] use a graph similar to the Abstract Meaning Representation (AMR) of the input, during training for an encoder architecture (with experiments in various classification tasks). Another work that relies on syntactic relations, by Lan and da Xue (2019) [14], introduce embeddings for syntactic features and then combine those embeddings,

and positional encodings. [We also infuse syntactic information but we rely on a single embedding and do not make changes to positional encodings nor to architectural elements of the language model.]

Wang et al. (2023) [15] construct specific "logic-aware" *embeddings* of the model's input sentence, which are combined with the token embeddings of the sentence to form the input embedding. [We instead feed the model with the interaction between the terms of the sentence via syntactic information and the semantic tree of the sentence; also, we encode information about the sentence structure along with the input text.]

In tasks related to code intelligence, Guo et al. (2022) [16] develop a pre-trained model for programming languages for code comprehension and generation tasks. The authors insert (in prompts) code comments and abstract syntactic trees (AST), with syntactic information such as declaration types and their nested relationships, to improve the representation of the code. To encode ASTs, represented by trees, the authors propose a mapping to transform the AST into a sequence that retains all the tree information and can be used as input to improve the code representation. The authors pre-train the language model.

We now add a few comments about the (rather large) literature on natural language to SQL translation, referred to as NL2SQL. Early efforts in NL2SQL were rule-based [17, 18], but now the best systems rely on deep networks trained with large datasets such as WikiSQL [19] and Spider [20]. However, transformers still struggle with phrasal structures, for instance ones that carry user intentions with multiple particular restrictions (that is, different adjectives for instance modifying different nouns). These difficulties are particularly acute in languages other than English for which datasets with examples are small. An alternative to this problem could be the use of a translation module for the user question to English, and then resort to NL2SQL in English. Nevertheless, this approach would bind the user to an external translation API or to a translation model from a given language to English, burdening the overall translation. In addition, José and Cozman (2021) [21] and Jose and Cozman (2023) [22] have found that a single multilingual transformer works better than a translation to English (followed by a English-trained transformer), for a number of source languages.

Efforts have been made in extending NL2SQL datasets to other languages besides English. Dou et al. (2023) [23] presented a dataset for the task covering seven languages and [22] broaden Spider to cover Spanish, Portuguese and French. [24] manually translated the also the Spider dataset to Chinese. In [25] and [26] Russian and Arabic datasets, respectively, have been developed for the task of translation of Natural Language to SQL. Several of these previous efforts used translated training data and found strong baselines that trained models with these datasets achieve, even though smaller, but yet comparable results to the ones obtained through training with English data only [24]. This shows the feasibility of training models directly using the target language as input to the language models, allowing one to unlock the many benefits of using an independent model for the translation from an input query in his native language to SQL.

There have been also works that employ parsers for NL2SQL. For instance, Yaghmazadeh et al.(2017) [27] and Iyer et al. (2017) [28] parse the input and map it to a

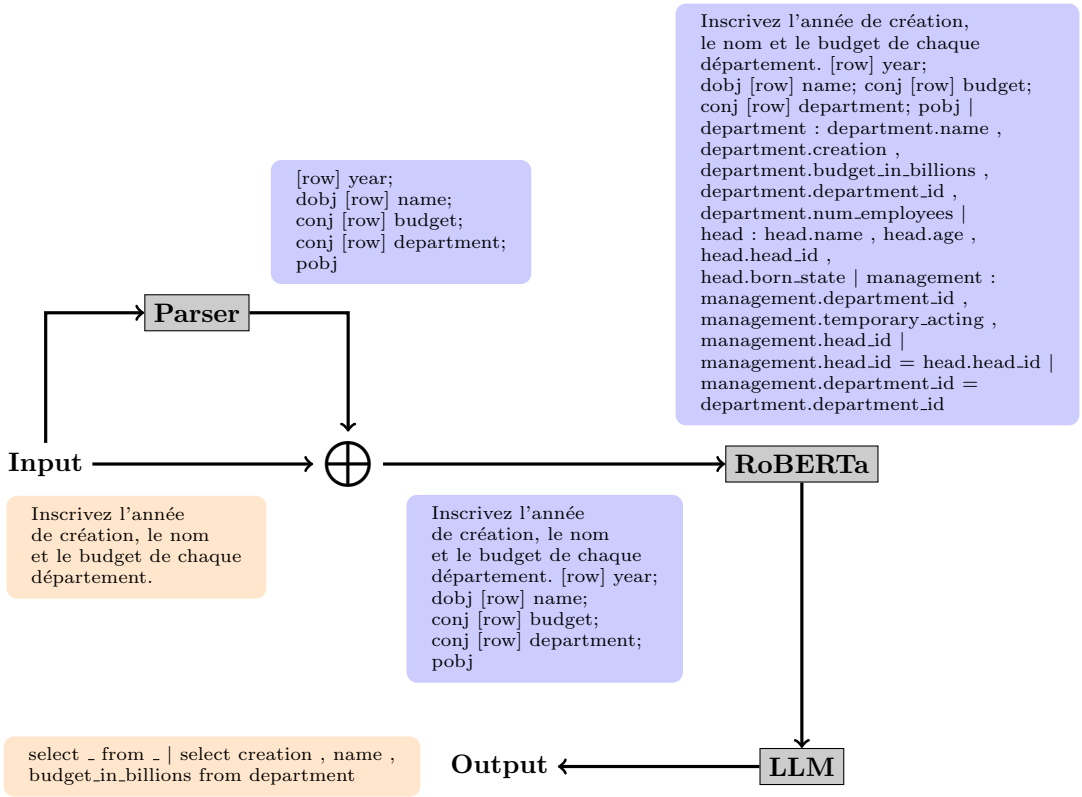


Fig. 1 The pipeline, from input to output.

special representation, such as a SQL skeleton, or even a proper SQL query. Neither of them actually apply linguistic analysis of the sentence structure. On the other hand, Li and Jagadish (2014) [17], even though relying on human feedback for the generated queries, make use of the phrase's morphological tree as an intermediate representation of the user question to generate by rule-mapping the SQL query, not using transformers. Neither of these approaches infuse formal phrase structure into transformers, nor use the input syntax or semantics, as we do in our method.

3 Infusing Syntactic and Semantic Information: A Proposal

In tasks where a little modification in the input phrase can produce a perceptive difference in the output, it is important for the model to capture the relations between the terms of the phrase as to generate the correspondent output, reflecting them, and consequently, reflecting the user's intention that they convey as a whole. For instance, in the task of translation from natural language to natural language, adjectives are supposed to only modify corresponding nouns, and such connections must be respected

in translated output. However, transformers are not always capable of interpreting correctly these relations, resulting in flawed outputs [1]. The same behavior is observed in translating natural language requests to SQL queries, in which the model must identify to which term a particular restriction refers to generate the correct SQL answer, for instance.

Intuitively, one might think that syntactic information is bound to improve LMs; however, it is not easy to see how to positively infuse at low computational cost. We explore a direct strategy where syntactic information is attached as part of the LM input during training (and may be attached as part of the input during inference as well).

To do so, we concatenate syntactic information with the raw input text. We use such enlarged input during training and inference, even though it is possible to produce outputs without enlarging the input (with syntactic and semantic information) during inference. Thus we do not have to change the architecture of the LM of choice.

The resulting pipeline is depicted in Figure 1: the input goes through syntactic and semantic analysis, and then it is concatenated with the result of that analysis; the concatenated text along with the relevant schema goes to the LM.

We rely on two types of information about the phrase: dependency trees [29], representing the syntax, and *Abstract Meaning Representation* (AMR) data, for the semantics. The AMR is a semantic representation framework that captures the meaning of a sentence through a directed acyclic graph where nodes correspond to concepts or entities, and edges represent various relationships between them. Concepts are typically abstract and language-independent in AMR, allowing for a higher level of semantic generalization. The relationships between concepts capture roles, attributes, and dependencies.

As a running example, take an input sentence as follows for a translation from French to SQL:

Inscrivez l'année de création, le nom et le budget de chaque département.[1]

The text produced out of the dependency tree for this input is:

Inscrivez ROOT Inscrivez PROPN [année, .]
l' det année NOUN []
année obj Inscrivez PROPN [l', création]
de case création NOUN []
création nmod année NOUN [de, ,, nom]
, punct création NOUN []
le det nom NOUN []
nom appos création NOUN [le, budget]
et cc budget NOUN []
le det budget NOUN []
budget conj nom NOUN [et, le, département]
de case département NOUN []
chaque det département NOUN []
département nmod budget NOUN [de, chaque]
. punct Inscrivez PROPN []

[1] In English: "List the creation year, name and budget of each department."

In this representation, the syntactic function in the phrase is written for each term of the phrase, followed by the term to which it refers to (to which it is dependent to in the relation, its "syntactic head"), its Part of Speech tag (morphology) and the terms that depends on it ("syntactic dependents").

However, relations such as determinants, that do not significantly affect meaning, are also included in this representation. Because we are only interested in teaching the model the syntactic relations that truly carry meaning, we only provide the model with a particular group of those relations: subjects, objects and conjunctions.

To translate a dependency tree into running text, we follow a proposal by Li et al. (2021) [30]: for each of these particular relations, we initiate the sentence with [row], followed by the term that plays the syntactic function and and the name of the syntactic function, separated by ";". For this running example, the prompt for the language model, following this method, is :

> Inscrivez l'année de création, le nom et le budget de chaque département. [row] year; dobj [row] name; conj [row] budget; conj [row] department; pobj

To infuse semantic information, we turn AMR graphs to text by removing spaces and relying on AMR brackets to separate the parser's output, so as to teach the model how to make the correct associations in PropBank notation [31]. For instance, if we have the same input sentence as above, the running text produced out of the AMR analysis is:

> (l / list - 01 :ARG1 (a / and :op1 (y / year :time - of (c / create - 01 :ARG1 (d / department :mod (e / each)))) :op2 (n / name :poss d) :op3 (b / budget :poss d)))

When combining it with the target sentence, we have:

> Inscrivez l'année de création, le nom et le budget de chaque département. (l / list - 01 :ARG1 (a / and :op1 (y / year :time - of (c / create - 01 :ARG1 (d / department :mod (e / each)))) :op2 (n / name :poss d) :op3 (b / budget :poss d)))

Finally, we have explored the impact of different ways of prompting to the model, through syntax and semantics, in independent experiments, but we also thought it would be interesting to see how the model would behave with the interaction between these two types of information about the phrase. Thus, we also presented to the models phrases that combine the two formats above. In order to separate the semantic representation from the syntactic structure, as we did to separate the syntactic terms, we include before the linearized AMR graph the tag "[AMR]". For the same input sentence, the resulting text combining syntax and semantics is:

> Inscrivez l'année de création, le nom et le budget de chaque département. [row] year; dobj [row] name; conj [row] budget; conj [row] department; pobj [AMR] (l / list - 01 :ARG1 (a / and :op1 (y / year :time - of (c / create - 01 :ARG1 (d / department :mod (e / each)))) :op2 (n / name :poss d) :op3 (b / budget :poss d)))

4 Experiments

Our experiments investigate whether syntactic and semantic information are valuable when translating natural language to SQL. This task is rather sensitive: a small mistake in input understanding may lead to seriously incorrect SQL queries. Thus one might expect linguistic analysis to be valuable.

In a sense, we hope to impart some linguistic reasoning into a model by inserting relevant information, and this should be more effective in languages with less available resources. For instance, in English we expect significant linguistic skills to be already absorbed by large language models.[2] We thus focus on languages with less support than English.

The dataset that we use across all experiments, for better comparison between techniques, is Spider [20], a popular corpus in NL2SQL. We used a translated version of Spider for each language we tested; each translated version was used across all tests for the corresponding language, so that relative performance results are meaningful within each language. In any case, the quality of translations is discussed later in this section. We should note that, as the Spider test set was not publicly available when many of the translations were done, we employed the development set for validation. Also, we note that there are recent corpora, such as BIRD [32], that are quite challenging for LLMs. We decided to stay with Spider for two reasons. First, there are several validated translations for Spider, in some cases with human validation; recent corpora do not have that. Second, Spider is still quite challenging in languages other than English, and those languages are our main focus.

Dependency trees and AMR graphs were generated by the Spacy parser[3] and amrlib[4], respectively. We evaluated our models using exact-set-match (ESM) and execution accuracy (EXA). These metrics are rather complementary, as exact-set-match may overlook some false negatives (one query might be the answer, even though not the golden one), whereas execution accuracy may overlook some false positives (a value might be equal to the one generated by the golden query, nonetheless by another term in the table, not the one referenced by the golden query). Moreover, to understand the individual impact of syntactic and semantic information of the input in a model performance and its relevance during training (and their effect compared to a simple prompt), we trained, for all languages and for all transformer architectures, a control model with the raw corresponding (translated) version of Spider, a model with the syntactic information, a model with the semantic information infused and a model with the two types of knowledge combined. Using this strategy, we are able to compare the results for training without any added linguistic information, solely with syntax infused, solely with semantics infused, and with the two pieces of information infused.

We investigated how our approach impacted two different language model architectures: encoder-decoder (Section 4.1) and decoder-only (Section 4.2).

[2] Intuitively, and perhaps a bit theatrically: if a person knows little about a language, then getting some exposure to syntactic and semantics lessons may be of significant help; but if a person has years of exposure to a language, syntactic and semantic lessons may not be as relevant for basic understanding of the language.

[3] https://spacy.io/api/dependencyparser

[4] https://github.com/bjascob/amrlib

4.1 Encoder-decoder

To fine-tune encoder-decoder architectures, we adopted the RESDSQL framework for NL2SQL [33]. In this framework, a trained encoder model (RoBERTa [34]), selects the most relevant tables and columns given the user's request, and presents only them, with the user question, to the transformer. In RESDSQL, the language model used to generate SQL first decodes the SQL structure, without assigning values, and then the language model generates the complete SQL output (with values). We make *no* changes to that framework, only to the tokens that we send as input to it.

We ran our experiments with encoder-decoders for French, Chinese, Portuguese and Spanish. We can expect the encoder-decoder models we used to have been exposed to relatively little data on these latter languages as compared to English.[5] To investigate the impact of our method in a language that is heavily used when training language models, we also ran experiments with the original English version of Spider.

The Chinese version of Spider we used has been translated by [24]. This translated dataset was carefully produced with the work of two NLP researchers and one computer science student. For complex phrases, the translator was allowed to rephrase the English question as to be able to make the most natural Chinese translation.

The French, Portuguese and Spanish versions of the Spider dataset have been produced by Jose and Cozman (2023) [22]. For each language, the authors employed automatic translation of the English phrases using the Google Translation API, followed by manual evaluation and revision. As automatic translation may distort contextual expressions, we examined the quality of that process: we run a detailed evaluation of the Portuguese translation of Spider, given that Portuguese is the lowest-resource language among the ones we dealt with and therefore is the most likely to present a challenge to the automatic translator (as one can surmise from Footnote 2). The evaluation process is summarized in the Technical Appendix.

We employed two different sequence-to-sequence models for fine-tuning, in order to verify the generalizability of the method, for French, Portuguese and Spanish: BART-base [35] and T5-base [36]. We emphasize that we opted in this set of experiments to use the monolingual versions of T5 and BART, not their multilingual models, in order to simulate the most adverse scenario one might encounter when choosing a model for a task, in which among the best possible models none of them has been trained with the language used in the application. For instance, mT5 [37] was not trained with a dataset that contains Croatian; therefore, despite mT5 being multilingual, a Croatian user would still face the challenge to accept to use a model that have not seen his native and intended use language before in its pre-training (as T5 has not seen as much French, Portuguese or Spanish, the languages we use in our experiments). In this way, we can measure the impact of our proposal in challenging scenarios.

For the Chinese models we did use mT5, as T5-base, the model with the best results for French, Portuguese and Spanish in our experiments, was too weak with Chinese (0.094 for ESM and 0.251 for EXA, scores prohibitively low to make any

[5] For instance, in the most recent CommonCrawl corpus (https://commoncrawl.org/), 43.79% of all texts are in English, while Chinese, Spanish and French appear respectively with 5.14%, 4.56% and 4.22% of texts, and Portuguese appears with meager 2.12% of texts.

safe conclusions around the impact of our method). As indicated later, the infusion procedure again benefits the translation, thus complementing the results above.

For fine-tuning the base models, the experiments were conducted on 2 GPUs NVidia GeForce RTX 3090 24 GB. For the specific fine-tuning of a T5 with 3 billion parameters for Portuguese and French a single A100 GPU with 80 GB was used. All models were trained using the AdaFactor optimizer, with a learning rate of 1e-4, batch size of 8 and gradient descent after two steps.

Following the fine-tuning strategy described in Section 4, we fine-tuned the three language models used, for each language, for each type of information (including the prompt without any information). Thus we have generated eight distinct language models per natural latin language (four BART and four T5) and four (mT5) for the Chinese language. We conducted this set of experiments both for 32 and for 128 training epochs for the encoder-decoder architectures. With these two numbers of training epochs, operating with the same data and language model, we were able to evaluate the relevance of the information presented to the progress of the training process itself.

4.1.1 Results and analysis

Table 1 (top) conveys the results for the translations for the French, Portuguese and Spanish corpus for 32 epochs, while Table 1 (bottom) presents the results of the models trained for 128 epochs with French, Spanish and Portuguese, respectively. In turn, Table 2 presents the results of the models trained with CSpider. The "ESM" and "EXA" columns contain values of exact-set match and execution accuracy respectively. The vESM and vEXA columns contain, respectively, the relative variation between the syntactic/semantic (or combined information) model's exact-set match accuracy and the control model's exact-set match accuracy, and the relative variation between the syntactic/semantic (or combined information) model's execution accuracy and the control model's execution accuracy, in percentages.

The tables show that the infusion of syntactic and semantic information is valuable during training. Models that were fed during training with the input phrase syntax or semantic structure displayed higher performance than the corresponding models without any linguistic information, regardless of the language model, for every natural language we tried. Also, this can be verified both in the lower number of epochs and in the higher number of epochs for the encoder-decoder architectures. In most experiments, the models that received semantic information had the higher scores. Thus we can see that syntactic and semantic information lead to better scores.

In addition, syntactic and semantic information accelerate training. Even when trained only for 32 epochs, models with syntactic or semantic information achieve higher results than models without any such information but trained with four times more epochs (that is, with 128 epochs). Therefore, the infusion not only attains better performance within a given number of training epochs, but it also attains a given performance with a smaller number of epochs.

We were in fact able to surpass the state of the art for Portuguese and French ([22]) when training the models using the T5 architecture with 3 billion parameters, NAT-SQL [38] as intermediate representation of the user's input and the corresponding SQL

Model	French				Spanish				Portuguese			
	ESM	**EXA**	**vESM**	**vEXA**	**ESM**	**EXA**	**vESM**	**vEXA**	**ESM**	**EXA**	**vESM**	**vEXA**
T5 without info	0.612	0.618	-	-	0.582	0.612	-	-	0.550	0.575	-	-
T5 with syntax	0.623	0.649	1.8	5.0	0.616	0.631	5.8	3.1	0.591	0.625	7.4	7.6
T5 with AMR	0.640	0.693	4.6	12.1	0.635	0.691	9.14	12.9	0.639	0.682	16.2	18.5
T5 with syntax + AMR	0.638	0.684	4.25	10.68	0.639	0.679	9.79	10.95	0.625	0.663	13.64	15.3
BART without info	0.199	0.199	-	-	0.199	0.199	-	-	0.182	0.181	-	-
BART with syntax	0.234	0.249	17.6	25.0	0.223	0.236	12.1	18.6	0.216	0.221	40.2	37.3
BART with AMR	0.215	0.225	8.0	13.0	0.242	0.262	21.6	31.7	0.222	0.245	40.2	52.4
BART with syntax + AMR	0.269	0.286	35.18	43.72	0.255	0.282	28.14	41.71	0.287	0.311	57.69	40.72

Model	French				Spanish				Portuguese			
	ESM	**EXA**	**vESM**	**vEXA**	**ESM**	**EXA**	**vESM**	**vEXA**	**ESM**	**EXA**	**vESM**	**vEXA**
T5 without info	0.618	0.632	-	-	0.591	0.618	-	-	0.586	0.602	-	-
T5 with syntax	0.657	0.673	6.3	6.5	0.631	0.636	6.8	2.9	0.630	0.649	7.4	7.9
T5 with AMR	0.669	0.700	8.2	10.8	0.653	0.687	10.5	11.2	0.653	0.687	11.4	14.1
T5 with syntax + AMR	0.652	0.695	5.5	9.97	0.651	0.708	10.15	14.56	0.649	0.697	10.75	15.78
BART without info	0.184	0.184	-	-	0.197	0.199	-	-	0.163	0.171	-	-
BART with syntax	0.238	0.240	29.3	30.4	0.218	0.220	10.7	10.6	0.218	0.222	33.1	29.9
BART with AMR	0.227	0.240	23.4	30.4	0.210	0.224	6.6	12.6	0.189	0.196	15.9	14.6
BART with syntax + AMR	0.279	0.299	51.63	62.5	0.276	0.289	40.1	45.23	0.285	0.305	74.85	78.36

Table 1 Model performance after training for 32 (top) and 128 (bottom) epochs.

Model	ESM	EXA	vESM	vEXA
T5 without info	0.094	0.251	-	-
mT5 without info	0.522	0.678	-	-
mT5 with syntax	0.550	0.710	5.36	4.72
mT5 with AMR	0.595	0.736	13.98	8.55
mT5 with syntax + AMR	0.594	0.723	13.79	6.64

Model	ESM	EXA	vESM	vEXA
mT5 without info	0.559	0.706	-	-
mT5 with syntax	0.582	0.739	4.11	4.67
mT5 with AMR	0.597	0.737	6.8	4.39
mT5 with syntax + AMR	0.625	0.747	11.81	5.81

Table 2 Chinese model performance after training for 32 (top) and 128 (bottom) epochs.

query and infusing semantic information (note that state of art results in the literature did not report execution accuracy). To be precise: while the best previous translator from Portuguese attained 0.687 in ESM, our system reached 0.752 in ESM and 0.804 in EXA; the previous best model for French attained 0.698 in ESM, our system reached 0.749 in ESM and 0.765 in EXA (previous results did not report execution-accuracy).

We also note that, comparing the performance of a model to its counterpart in the parallel group of training epochs (keeping the same architecture and for the same language but training for a different number of epochs), the models infused with syntactic or semantic information trained for 32 epochs had values for both metrics near the values from their counterparts trained for 128 epochs. This suggests an expressive gain in training efficiency, allowing results near the values one would get after training the model for 128 epochs only after 32 epochs of training with syntax or semantics. This presents a reduction in the time cost of training the model, as well as in the energy consumed for it.

We have also ran experiments using the original Spider dataset, in English, as can be seen in Table 3. We saw little improvement, and in some cases a decrease in performance, when prompts received syntactic and semantic information. For instance, the model T5-3B trained with AMR achieved exact set match of 0.688 and execution accuracy of 0.784, and the model without information had 0.714 and 0.787 for the respective metrics. We conjecture that a large language model trained with a vast number of textual data somehow internalizes all information that could be given through linguistic analysis (as we have discussed previously).

4.2 Decoder-only

We have also explored the effect of our approach for decoder-only models. Given the size and scope of current decoder-only models, we expect them to actually have good performance in many languages with lower resources than English (but not "really low" resources), such as the languages we used in the encoder-decoder experiments.

Model	ESM	EXA	vESM	vEXA
T5 without info	0.717	0.770	-	-
T5 with syntax	0.722	0.768	0.7	-0.26
T5 with AMR	0.708	0.752	-1.26	-2.34
T5 with syntax + AMR	0.700	0.756	-2.37	-1.82
BART with no info	0.303	0.326	-	-
BART with syntax	0.295	0.313	-2.64	-3.99
BART with AMR	0.310	0.327	2.31	0.31
BART with AMR + syntax	0.315	0.341	3.96	4.6

Model	ESM	EXA	vESM	vEXA
T5 without info	0.717	0.779	-	-
T5 with syntax	0.722	0.776	0.7	-0.39
T5 with AMR	0.714	0.778	-0.42	-0.13
T5 with syntax + AMR	0.719	0.769	0.28	-1.28
BART with no info	0.298	0.307	-	-
BART with syntax	0.290	0.293	-2.68	-4.56
BART with AMR	0.296	0.321	-0.67	4.56
BART with AMR + syntax	0.281	0.292	-5.7	-4.88

Table 3 English model performance after training for 32 (top) and 128 (bottom) epochs.

On the other hand, previous investigation has suggested that decoder-only models often struggle with low-resource languages [39], [40], [41].

We note that, even though linguistic tools are not common for less explored languages, efforts have been made in that direction [42, 43]. Heinecke and Shimorina (2022) [44] developed a AMR parser for low-resource Celtic languages. Datasets contemplating low-resource languages have also emerged [45].

The languages that we chose to use were Welsh, a low-resource language with an AMR-parser, and Guarani, a very low-resource language with scarce linguistic tools. Our goal here is to show that, *if semantic analysis were available for Welsh and Guarani*, its infusion would improve the translation process. To simulate the needed AMR-parsers, we used the information available from other languages discussed previously as infusion material (once we use the same base dataset, the phrase's intent is the same, as its AMR). We do not use syntactic information for Welsh and Guarani as syntax is very language-dependent.

We translated Spider using the NLLB model [46] to Welsh and Guarani. Accuracy of this translator is analyzed in the original paper, an effort to keep low-resource languages alive by translating them to/from English.

To select tables and columns to present along with the question and phrase's information as input to the LLM, we used an encoder model (RoBERTa) trained in the encoder-decoder experiment. Because we do not use syntactic information in

Model	ESM	EXA	vESM	vEXA
Mistral without info	0.482	0.469	-	-
Mistral with AMR	0.574	0.575	19.09	22.6

Table 4 Welsh model performance after training for 4 epochs.

Model	ESM	EXA	vESM	vEXA
Mistral without info	0.363	0.385	-	-
Mistral with AMR	0.590	0.603	62.53	56.62

Table 5 Guarani model performance after training for 4 epochs.

these experiments, we fine-tuned for each language a model without any information (control) and other with semantic information in the input. Therefore, we fine-tuned four LLMs, two for Guarani and two for Welsh.

As representative of the decoder-only architecture, we selected Mistral-7B-Instruct-v0.3 [6]. We have fine-tuned the model only for eight epochs, as its extensive pre-training enables it to achieve good results within few training epochs, while also saving time and energy consumption. To evaluate the impact of our approach to the progress of the training itself, as we did with the two different epochs fine-tuning in Section 4.1, we look at the loss of the two models for a given language during the whole fine-tuning using plots (shown later).

We used as computing infrastructure 4 GPUs NVidia GeForce RTX 4000 20 GB. The fine-tuning parameters were LoRA r = 64, alpha = 16 and dropout = 0.1. We also used 4-bit quantization, AdamW as optimizer with 2e-4 learning rate. We fine-tuned the model for 8 epochs, with batch size of 4, with 4 gradient accumulation steps.

4.2.1 Results and analysis

Tables 4 and 5 present results related to training with Welsh and Guarani versions of Spider (as explained, we only infused semantic information). As in the encoder-decoder experiments, models benefit from semantic information.

In Figure 2, we plot the loss curve for the fine-tuning of the Welsh models for the first two epochs, in order to better visualize the curves for fine-tuning without any information and with semantics. We note that semantic information led to lower final loss, and led in general to lower loss than the control model (prompted only with question and database schema). The loss curves for Guarani are in the Technical Appendix.

[6] https://huggingface.co/mistralai/Mistral-7B-Instruct-v0.3

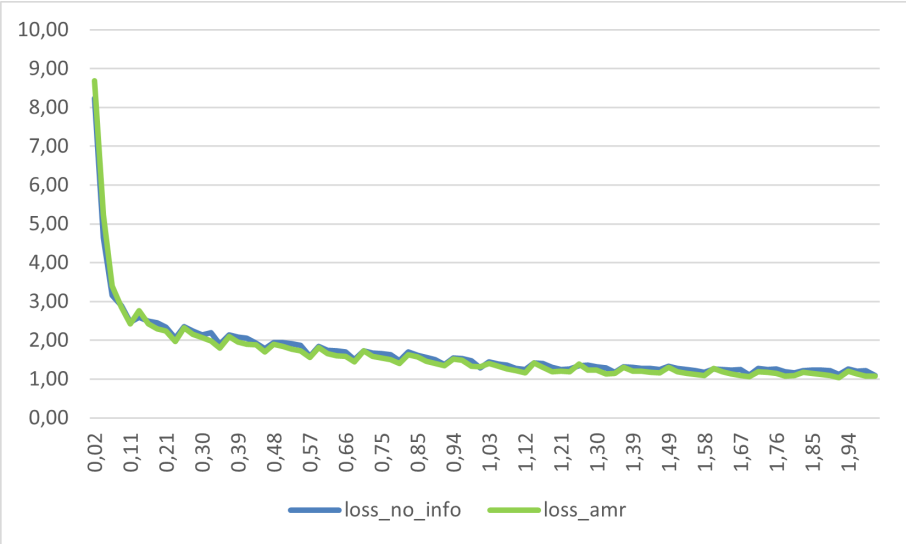


Fig. 2 Loss for fine-tuning the Welsh model, with semantic (green) and without information (blue).

5 Conclusion

In this paper we have a concise message: Syntactic and semantic information should be used together with LLMs in low resource scenarios! The former can improve the latter, at low cost, and in fact linguistic analysis can mitigate the need for ever larger annotated datasets. This sort of message has been pursued in previous work, but we feel that we have here described strategies that are simpler to apply, and that seem to be more effective. It should be emphasized that we do not require changes in inner components of transformers (such as positional embeddings, self attention layers, etc).

Our experiments show that syntactic and semantic infusion helps in NL2SQL from six natural languages, often in a smaller number of training epochs. The scheme with the best contribution was the one based on AMR graphs, even though dependency trees were also valuable. We close with two key points that our experiments indicate about our proposals.

- Effectiveness: the positive value of training transformers with syntactic and semantic information, at least for low resource scenarios, without retraining the models (linguistic analysis led models to attain higher performance than possible otherwise).
- Efficiency: syntactic and semantic information do accelerate the training process, enabling models to reach target performance levels in fewer training epochs.

Concerning future work, we suggest that other tasks in natural language processing should be investigated as to whether syntactic and semantic information lead to better performance. Another key (and complex) issue is whether LLMs learn, in some suitable way, to do linguistic analysis when exposed to "sufficiently large" resources — that is, to understand whether there are regimes where explicit addition of syntactic and semantic information does not make a difference as such information is already

internally incorporated. Obtaining insight on this issue would help one better design systems by balancing the resources devoted to a large language model against the level of linguistic information extracted from input and infused into the language model.

Supplementary information. The article has no supplementary files.

Acknowledgements. This work was supported by the Ministry of Science, Technology and Innovation, with resources of Law N. 8.248, of October 23, 1991, within the scope of PPI-SOFTEX, coordinated by Softex and published as Residence in TIC 13, DOU 01245.010222/2022-44. The authors of this work would like to thank the Center for Artificial Intelligence (C4AI-USP) and the support from the São Paulo Research Foundation (FAPESP grant 2019/07665-4) and from the IBM Corporation. F. G. C. was partially supported by CNPq grants 312180/2018-7 and 305753/2022-3. The authors also thank support by CAPES -Finance Code 001.

Declarations

Conflict of interest. The authors declare no competing interests.

Funding. The work has been supported by the Ministry of Science, Technology and Innovation, with resources of Law N. 8.248, of October 23, 1991, within the scope of PPI-SOFTEX, coordinated by Softex and published as Residence in TIC 13, DOU 01245.010222/2022-44; it has also been supported by the Center for Artificial Intelligence USP/IBM/FAPESP (C4AI), jointly funded by the São Paulo Research Foundation (FAPESP grant 2019/07665-4) and by the IBM Corporation. The last author was partially supported by CNPq grants 312180/2018-7 and 305753/2022-3. We acknowledge support also by CAPES - Finance Code 001.

Consent for publication. The authors have read and agreed to the terms of publication.

Data availability. The datasets (Spider) used for training and validating the models in Chinese, French, Portuguese and Spanish are public and available at links in their respective publications. The modifications made to these datasets (such as inclusion of semantic information) relied on the parsing of the Spider dataset's sentences (train and validation splits) and appending the results to the sentences.

Code availability. The code for fine-tuning T5 and BART is the framework of RESDSQL, which is available by its corresponding authors at https://github.com/RUCKBReasoning/RESDSQL. The code that we developed for fine-tuning Mistral-7B-Instruct-v0.3 is available at https://github.com/antonlabate/semantic-infusion.

Ethics approval and consent to participate. Not applicable.

Materials availability. Not applicable.

Appendix A Training Models for 32 and 128 Epochs

As noted in the main text, we trained our models with corpora containing queries annotated with syntactic and semantic information. Here we show that one of the benefits of infusing linguistic information in the training process is that it increases the speed of training for the encoder-decoder experiment. That is, within a few epochs, we can achieve results that are close to the ones we achieve after more training epochs without linguistic information (hence letting users with a constrained time scheduled to train a model only with a few epochs, using our techniques, and yet achieve results similar to the ones they would achieve if the model was trained for more epochs without our techniques).

This can be seen by looking at data for a mT5 model trained with CSpider with AMR for 32 and for 128 epochs. The model trained for 32 epochs achieved 0.595 of exact-set match and 0.736 for execution-accuracy, while the model trained for 128 epochs presented exact-set match of 0.597 and 0.737 for the former and latter metrics (that is, very similar results). More importantly for the purposes of this appendix, there was less computation required in the former setting: In Table C1, we present the time required for fine-tuning a mT5 base using CSpider and semantic information infused for 32 and 128 epochs, with the same hyper-parameter settings.

These experiments were run with a computer with one GPU NVidia GeForce RTX 3090 24 GB, with a consumption of approximately 330 W, with the same parameters. In the table, we also present energy consumption during fine-tuning. So, if users have resource constraints, such as time and energy, they are able to fine-tune a model using our method for 32 epochs, and still obtain results near the ones they would get after fine-tuning for 128 epochs.

Appendix B Analysis of the Errors in Automatic Translation

As noted in the main text, in order to get a grasp of the type of errors we might encounter with the automatic translations of the Spider corpus, and hence to get a sense of their impact to the phrase and for our model trained with them in general, we had a professional translator analyze translated queries in Portuguese (a worst case analysis, given that Portuguese is the lowest resource language among the ones we experimented with).

In addition the experiments reported in the main text, we also asked the professional translator to comment and point out corrections for the errors she found in 300 sampled queries and their translations. We also asked the translator whether, despite the incorrect translations, the answer to the automatic translated query would be the same as the answer for her version of the translated query. With this approach, we were able to assess the impact of translation errors in the final result of interest. In Table C2, we present, segmented by query difficulty level, the proportion of queries in each level that, regardless of not matching the golden translation, still led to the same answer (according to the translator).

Epochs	Time for fine-tuning (min)	Energy consumption (kJ)
32	147	2910.60
128	593	11741.40

Table C1 Time and energy consumed for training a mT5 with CSpider database for 32 and 128 epochs.

As an example, for the query "What are names of the movies that are either made after 2000 or reviewed by Brittany Harris?", the translation to Portuguese was "Quais são os nomes dos filmes feitos depois de 2000 ou resenhas por Brittany Harris?". The translator pointed out that "resenhas" should be substituted for a better translation to "avaliados". However, the answer for both queries would still be the same. To illustrate a different situation, the query "What is the number of different class sections offered in the course ACCT-211?" was translated to "Qual é o número de diferentes seções de aula oferecidas no curso ACCT-211?". The translator indicated that class sections should be translated to "grupos de estudo", which makes a fundamental difference in the possible answers for the query — therefore the automatically translated query did not have the same answer as the human translated query.

Even though some translated queries had flaws in particular words or expressions, the vast majority of them actually preserved the fundamental question that was asked. We can view this in Table C2, where most of the queries evaluated, independently of their translation score, still led to the same answer as the human translation. Therefore, the translation did not introduce fundamental problems to the corpus; even though the automated translation has flaws, its impact in the overall analysis is rather small.

Appendix C Loss Curves for Experiments with Welsh and Guarani Languages

We present in Figure C1 the loss curves for the fine-tuning of Mistral-7B-Instruct-v0.3, for 8 epochs and for both Welsh and Guarani languages.

For both languages, we report the loss of fine-tuning the model without any infused syntactic/semantic information and with semantic information. Fine-tuning with semantic information in the prompt eventually attains significantly lower loss (as shown in tables in the main text); during training differences are minor.

References

[1] Leivada, E., Murphy, E., Marcus, G.: DALL-E 2 Fails to Reliably Capture Common Syntactic Processes (2022)

[2] Yao, Y., Koller, A.: Structural generalization is hard for sequence-to-sequence models (2022)

[3] Punyakanok, V., Roth, D., Yih, W.-t.: The importance of syntactic parsing and inference in semantic role labeling. Computational Linguistics **34**(2), 257–287

Query difficulty	Percentage with same answer
Easy	93%
Medium	100%
Hard	99%
Extra	100%

Table C2 Evaluation of translated Portuguese version of the Spider corpus by professional translator: percentage of translated queries that (despite possible specific flaws in the translation) led to the same answer as the original query in English.

(2008)

[4] Søgaard, A., Goldberg, Y.: Deep multi-task learning with low level tasks supervised at lower layers. In: Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers), pp. 231–235. Association for Computational Linguistics, Berlin, Germany (2016). https://doi.org/10.18653/v1/P16-2038 . https://aclanthology.org/P16-2038

[5] Shiv, V., Quirk, C.: Novel positional encodings to enable tree-based transformers. Advances in neural information processing systems **32** (2019)

[6] Qian, P., Naseem, T., Levy, R., Astudillo, R.F.: Structural Guidance for Transformer Language Models (2021)

[7] Bai, J., Wang, Y., Chen, Y., Yang, Y., Bai, J., Yu, J., Tong, Y.: Syntax-bert: Improving pre-trained transformers with syntax trees. arXiv preprint arXiv:2103.04350 (2021)

[8] Zanzotto, F.M., Santilli, A., Ranaldi, L., Onorati, D., Tommasino, P., Fallucchi, F.: KERMIT: Complementing transformer architectures with encoders of explicit syntactic interpretations. In: Webber, B., Cohn, T., He, Y., Liu, Y. (eds.) Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp. 256–267. Association for Computational Linguistics, Online (2020). https://doi.org/10.18653/v1/2020.emnlp-main.18 . https://aclanthology.org/2020.emnlp-main.18/

[9] Currey, A., Heafield, K.: Incorporating source syntax into transformer-based neural machine translation. In: Proceedings of the Fourth Conference on Machine Translation (Volume 1: Research Papers), pp. 24–33 (2019)

[10] Zhang, Z., Wu, Y., Zhou, J., Duan, S., Zhao, H., Wang, R.: Sg-net: Syntax-guided machine reading comprehension. Technical report (2020). www.aaai.org

[11] Zhang, Z., Wu, Y., Zhou, J., Duan, S., Zhao, H., Wang, R.: Sg-net: Syntax guided transformer for language representation. IEEE Transactions on Pattern Analysis and Machine Intelligence **44**(6), 3285–3299 (2022) https://doi.org/10.1109/

TPAMI.2020.3046683

[12] Yasunaga, M., Bosselut, A., Ren, H., Zhang, X., Manning, C.D., Liang, P., Leskovec, J.: Deep bidirectional language-knowledge graph pretraining. In: Neural Information Processing Systems (NeurIPS) (2022)

[13] Zhang, Z., Wu, Y., Zhao, H., Li, Z., Zhang, S., Zhou, X., Zhou, X.: Semantics-aware bert for language understanding. In: AAAI Conference on Artificial Intelligence, vol. 34, pp. 9628–9635 (2020)

[14] Lan, M., Xue, H.D.S.F.: International Conference on Asian Language Processing (IALP), p. 508 (2019)

[15] Wang, B., Huang, Q., Deb, B., Halfaker, A., Shao, L., McDuff, D., Awadallah, A.H., Radev, D., Gao, J.: Logical transformers: Infusing logical structures into pre-trained language models. In: Findings of the Association for Computational Linguistics: ACL 2023, pp. 1762–1773. Association for Computational Linguistics, Toronto, Canada (2023). https://doi.org/10.18653/v1/2023.findings-acl.111 . https://aclanthology.org/2023.findings-acl.111

[16] Guo, D., Lu, S., Duan, N., Wang, Y., Zhou, M., Yin, J.: UniXcoder: Unified cross-modal pre-training for code representation. In: Muresan, S., Nakov, P., Villavicencio, A. (eds.) Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pp. 7212–7225. Association for Computational Linguistics, Dublin, Ireland (2022). https://doi.org/10.18653/v1/2022.acl-long.499 . https://aclanthology.org/2022.acl-long.499/

[17] Li, F., Jagadish, H.V.: Nalir: an interactive natural language interface for querying relational databases. In: Proceedings of the 2014 ACM SIGMOD International Conference on Management of Data. SIGMOD '14, pp. 709–712. Association for Computing Machinery, New York, NY, USA (2014). https://doi.org/10.1145/2588555.2594519 . https://doi.org/10.1145/2588555.2594519

[18] Saha, D., Floratou, A., Sankaranarayanan, K., Minhas, U.F., Mittal, A.R., Özcan, F.: Athena: an ontology-driven system for natural language querying over relational data stores. Proc. VLDB Endow. **9**(12), 1209–1220 (2016) https://doi.org/10.14778/2994509.2994536

[19] Zhong, V., Xiong, C., Socher, R.: Seq2sql: Generating structured queries from natural language using reinforcement learning. arXiv preprint arXiv:1709.00103 (2017)

[20] Yu, T., Zhang, R., Yang, K., Yasunaga, M., Wang, D., Li, Z., Ma, J., Li, I., Yao, Q., Roman, S., Zhang, Z., Radev, D.: Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task (2019)

[21] José, M.A., Cozman, F.G.: mrat-sql+ gap: a portuguese text-to-sql transformer. In: Intelligent Systems: 10th Brazilian Conference, pp. 511–525 (2021). Springer

[22] Jose, M.A., Cozman, F.G.: A multilingual translator to sql with database schema pruning to improve self-attention. International Journal of Information Technology **15**(6), 3015–3023 (2023)

[23] Dou, L., Gao, Y., Pan, M., Wang, D., Che, W., Zhan, D., Lou, J.-G.: Multispider: towards benchmarking multilingual text-to-sql semantic parsing. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 37, pp. 12745–12753 (2023)

[24] Min, Q., Shi, Y., Zhang, Y.: A pilot study for Chinese SQL semantic parsing. In: Inui, K., Jiang, J., Ng, V., Wan, X. (eds.) Conference on Empirical Methods in Natural Language Processing, pp. 3652–3658. Association for Computational Linguistics, Hong Kong, China (2019). https://doi.org/10.18653/v1/D19-1377 . https://aclanthology.org/D19-1377

[25] Bakshandaeva, D., Somov, O., Dmitrieva, E., Davydova, V., Tutubalina, E.: Pauq: Text-to-sql in russian. In: Findings of the Association for Computational Linguistics: EMNLP 2022, pp. 2355–2376 (2022)

[26] Almohaimeed, S., Almohaimeed, S., Al Ghanim, M., Wang, L.: Ar-spider: Text-to-sql in arabic. In: ACM/SIGAPP Symposium on Applied Computing, pp. 1024–1030 (2024)

[27] Yaghmazadeh, N., Wang, Y., Dillig, I., Dillig, T.: Sqlizer: Query synthesis from natural language. Proceedings of the ACM on Programming Languages **1** (2017) https://doi.org/10.1145/3133887

[28] Iyer, S., Konstas, I., Cheung, A., Krishnamurthy, J., Zettlemoyer, L.: Learning a Neural Semantic Parser from User Feedback (2017)

[29] Jurafsky, D., Martin, J.H.: Speech and Language Processing An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition Third Edition Draft, (2023)

[30] Li, A.H., Ng, P., Xu, P., Zhu, H., Wang, Z., Xiang, B.: Dual reader-parser on hybrid textual and tabular evidence for open domain question answering. arXiv preprint arXiv:2108.02866 (2021)

[31] Palmer, M., Gildea, D., Kingsbury, P.: The Proposition Bank: An Annotated Corpus of Semantic Roles. Computational Linguistics **31**(1), 71–106 (2005) https://doi.org/10.1162/0891201053630264 https://direct.mit.edu/coli/article-pdf/31/1/71/1798172/0891201053630264.pdf

[32] Li, J., Hui, B., Qu, G., Yang, J., Li, B., Li, B., Wang, B., Qin, B., Geng, R., Huo,

N., et al.: Can llm already serve as a database interface? a big bench for large-scale database grounded text-to-sqls. Advances in Neural Information Processing Systems **36** (2024)

[33] Li, H., Zhang, J., Li, C., Chen, H.: RESDSQL: Decoupling schema linking and skeleton parsing for text-to-sql. In: AAAI Conference on Artificial Intelligence, vol. 37, pp. 13067–13075 (2023)

[34] Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L., Stoyanov, V.: RoBERTa: A robustly optimized bert pretraining approach. arXiv preprint arXiv:1907.11692 (2019)

[35] Lewis, M., Liu, Y., Goyal, N., Ghazvininejad, M., Mohamed, A., Levy, O., Stoyanov, V., Zettlemoyer, L.: BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension (2019)

[36] Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., Liu, P.J.: Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer (2023)

[37] Xue, L., Constant, N., Roberts, A., Kale, M., Al-Rfou, R., Siddhant, A., Barua, A., Raffel, C.: mT5: A massively multilingual pre-trained text-to-text transformer (2021). https://arxiv.org/abs/2010.11934

[38] Gan, Y., Chen, X., Xie, J., Purver, M., Woodward, J.R., Drake, J., Zhang, Q.: Natural sql: Making sql easier to infer from natural language specifications. arXiv preprint arXiv:2109.05153 (2021)

[39] Lai, V.D., Ngo, N.T., Veyseh, A.P.B., Man, H., Dernoncourt, F., Bui, T., Nguyen, T.H.: ChatGPT Beyond English: Towards a Comprehensive Evaluation of Large Language Models in Multilingual Learning (2023). https://arxiv.org/abs/2304.05613

[40] Hangya, V., Saadi, H.S., Fraser, A.: Improving low-resource languages in pre-trained multilingual language models. In: Goldberg, Y., Kozareva, Z., Zhang, Y. (eds.) Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing, pp. 11993–12006. Association for Computational Linguistics, Abu Dhabi, United Arab Emirates (2022). https://doi.org/10.18653/v1/2022.emnlp-main.822 . https://aclanthology.org/2022.emnlp-main.822/

[41] Bang, Y., Cahyawijaya, S., Lee, N., Dai, W., Su, D., Wilie, B., Lovenia, H., Ji, Z., Yu, T., Chung, W., Do, Q.V., Xu, Y., Fung, P.: A multitask, multilingual, multimodal evaluation of ChatGPT on reasoning, hallucination, and interactivity. In: International Joint Conference on Natural Language Processing and Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics, vol. 1, pp. 675–718 (2023). https://doi.org/10.18653/v1/2023.ijcnlp-main.45 . https://aclanthology.org/2023.ijcnlp-main.45

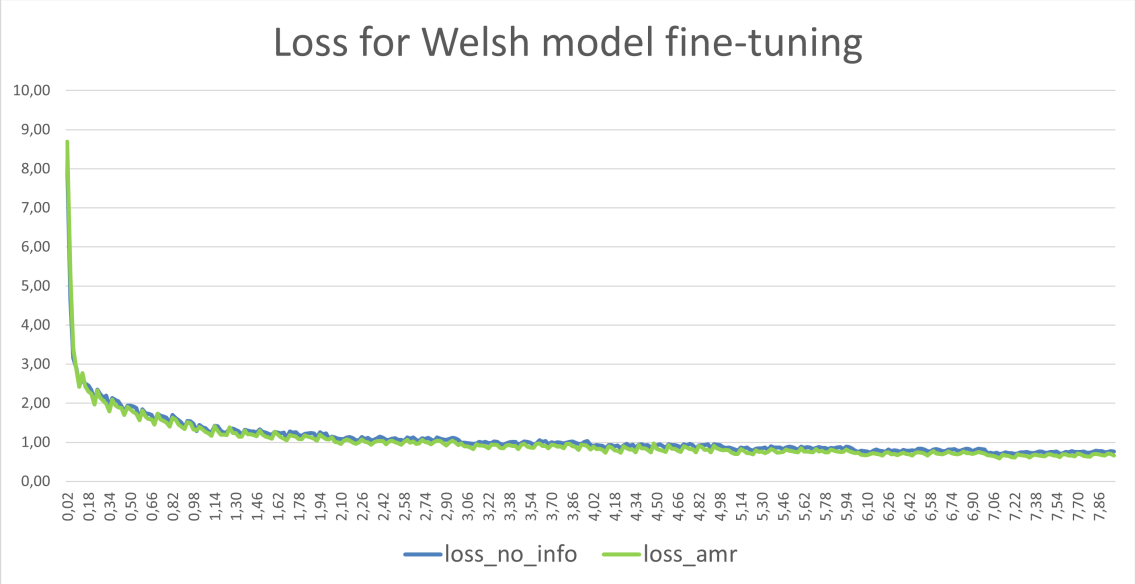
[42] Cai, D., Li, X., Ho, J.C.-S., Bing, L., Lam, W.: Multilingual AMR parsing with noisy knowledge distillation. In: Findings of the Association for Computational Linguistics: EMNLP 2021, pp. 2778–2789 (2021). https://doi.org/10.18653/v1/2021.findings-emnlp.237 . https://aclanthology.org/2021.findings-emnlp.237/

[43] Vanroy, B., Cruys, T.V.D.: Less is Enough: Less-Resourced Multilingual AMR Parsing (2024). https://github.com/BramVanroy/

[44] Heinecke, J., Shimorina, A.: Multilingual Abstract Meaning Representation for Celtic languages. In: Fransen, T., Lamb, W., Prys, D. (eds.) Proceedings of the 4th Celtic Language Technology Workshop Within LREC2022, pp. 1–6. European Language Resources Association, Marseille, France (2022). https://aclanthology.org/2022.cltw-1.1/

[45] Regan, M., Wein, S., Baker, G., Monti, E.: MASSIVE Multilingual Abstract Meaning Representation: A Dataset and Baselines for Hallucination Detection (2024). https://arxiv.org/abs/2405.19285

[46] Costa-jussà, M.R., Cross, J., Çelebi, O., Elbayad, M., Heafield, K., Heffernan, K., Kalbassi, E., Lam, J., Licht, D., Maillard, J., Sun, A., Wang, S., Wenzek, G., Youngblood, A., Akula, B., Barrault, L., Gonzalez, G.M., Hansanti, P., Hoffman, J., Jarrett, S., Sadagopan, K.R., Rowe, D., Spruit, S., Tran, C., Andrews, P., Ayan, N.F., Bhosale, S., Edunov, S., Fan, A., Gao, C., Goswami, V., Guzmán, F., Koehn, P., Mourachko, A., Ropers, C., Saleem, S., Schwenk, H., Wang, J.: No Language Left Behind: Scaling Human-Centered Machine Translation (2022). https://arxiv.org/abs/2207.04672



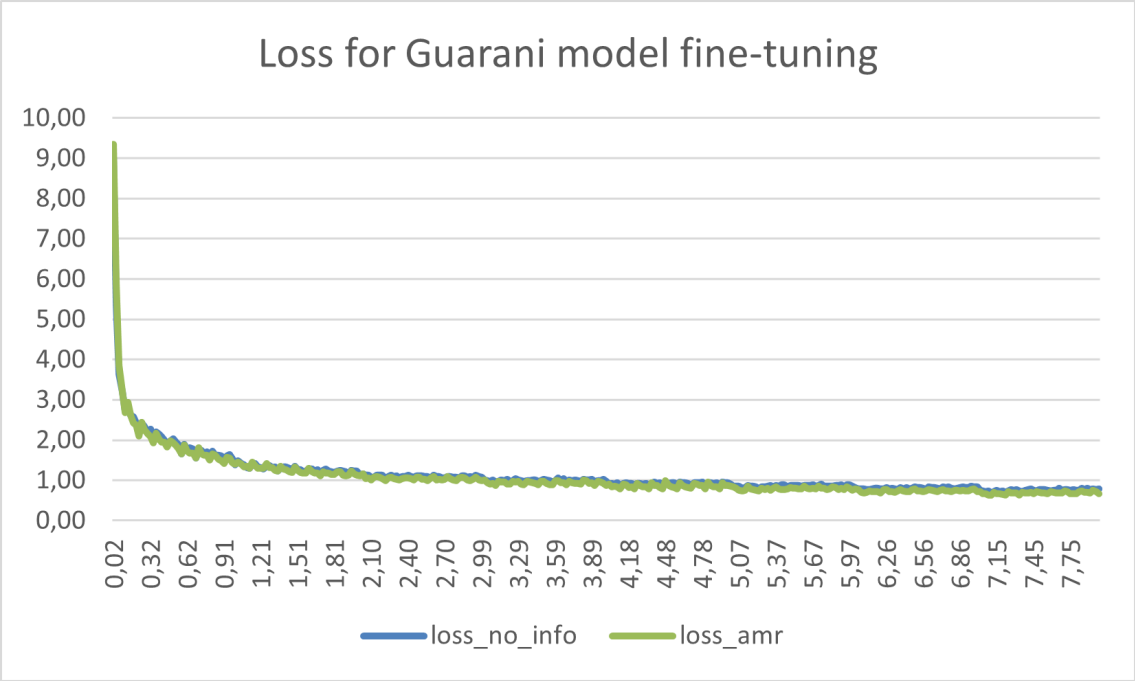


Fig. C1 Loss for fine-tuning the models for Welsh (top) and Guarani (bottom) languages, with different inputs. The loss is in y-axis, plotted against the epoch of the fine-tuning in the X-axis.